

P3971R1

std::rebind_cast - Generalised Element-Type Conversion for Containers and Uniform-Element Types

Published Proposal, 2026-04-30

This version:

<http://wg21.link/P3971>

Author:

[Daniel Towner](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes `std::rebind_cast`, a facility for converting containers and other uniform-element types to use a different element type while preserving structure. This enables generic programming over types whose element type can be meaningfully changed (sequence containers, `std::complex`, SIMD-like types, user-defined uniform-element types) using a single, named, value-producing cast. The name `rebind_cast` is chosen for consistency with the established `_cast` family (alongside `reinterpret_cast`, `bit_cast`, `saturating_cast`, `duration_cast`, etc.) and to keep a clean separation between the type-level trait `rebind_t` and the value-producing operation `rebind_cast`.

Table of Contents

1 Revision History

2 Motivation

2.1 Precedent in `std::simd`

2.2 Proposed Solution

3 Supported Types

3.1 Specification Principle

3.2 Standard Library Support

3.3 User-Defined Types

3.4 Excluded Types

4 Examples

4.1 Basic Usage

4.2 Generic Type Conversion

4.3 Complex Numbers

4.4 User-Defined Types

5 Design Alternatives

5.1 Range Adaptor Syntax

5.2 Tuple and Pair

5.3 Container Adaptors

5.4 Associative Containers

5.5 Naming

5.5.1 Consistency with the established `_cast` family

5.5.2 Comparison with other candidate names

5.5.3 Disambiguation from `std::simd::rebind`

5.6 Customisation Mechanism

5.7 Relationship Between `rebind_t` and `rebind_cast`

5.8 Relationship with `std::simd::rebind_t`

5.8.1 Coexistence

5.8.2 Equivalence guarantee

5.8.3 Customisation goes through `rebind_cast`, not via trait specialisation

5.8.4 Guidance for users

5.9 Value-Preserving Conversions

5.10 Duration and Units Types

6 Implementation Experience

7 Wording

- 7.1 Header `<utility>` synopsis additions
- 7.2 `[utility.rebind]` Generalised element-type conversion
 - 7.2.1 General
 - 7.2.2 Type alias `rebind_t`
 - 7.2.3 Customisation point object `rebind_cast`
 - 7.2.4 Relationship with `std::simd::rebind_t`
 - 7.2.5 Customisation by user-defined types
- 7.3 Feature-test macro

References

Informative References

§ 1. Revision History

R1

- Renamed the proposed facility from `std::rebind` to `std::rebind_cast`.
- Expanded §5.5 Naming with a comparison against the established `_cast` family and an explicit rejection of the bare `rebind` name.
- The type trait remains named `rebind_t` (preserving the precedent from `std::simd`); only the value-producing operation gains the `_cast` suffix.
- Updated all examples and wording to reflect the new name.

R0

- Initial revision

§ 2. Motivation

Modern C++ provides powerful facilities for generic programming, but lacks a uniform way to change the element type of containers and container-like types. Consider a simple requirement: convert a container of `float` into the corresponding container of `double`, without the caller having to know what kind of container it is.

For `std::vector`, this is straightforward using constructor-based conversion:

```
std::vector<double> widen(const std::vector<float>& data) {  
    return std::vector<double>(data.begin(), data.end());  
}
```

However, making this work for `std::array` requires completely different code:

```
template<std::size_t N>  
std::array<double, N> widen(const std::array<float, N>& data) {  
    std::array<double, N> result;  
    std::copy(data.begin(), data.end(), result.begin());  
    return result;  
}
```

For `std::complex`, yet another approach is needed:

```
std::complex<double> widen(const std::complex<float>& data) {  
    return std::complex<double>(data);  
}
```

Each type requires specialised knowledge of its conversion mechanisms. Attempting to write a single generic function fails:

```
template<typename Container>  
auto widen_to_double(const Container& data) {  
    using T = typename Container::value_type; // Some container types may r  
  
    // How do we create Container<double> from Container<float>?  
    // - vector: range constructor  
    // - array: manual copy with known size  
    // - complex: direct construction  
    // - user types: ???  
  
    // No uniform solution exists  
}
```

§ 2.1. Precedent in `std::simd`

`std::simd` recognised this problem and introduced `rebind_t` as a type trait to convert the type `basic_vec<T, Abi>` to `basic_vec<U, Abi>`. This works well for `simd`, but the same need exists across containers, `std::complex`, and user-defined uniform-element types.

Recent discussion of `simd` casting utilities [P3445R0] raised questions about whether such facilities should be generalised beyond `simd` to support other types like containers and units. This proposal explores that generalisation.

§ 2.2. Proposed Solution

We propose two complementary facilities:

1. `rebind_t<U, T>`: a type trait that computes the result type of rebinding the structure of `T` to use elements of type `U`.
2. `rebind_cast<U>(obj)`: a customisation point object that performs the element-wise conversion and produces a new object of type `rebind_t<U, decltype(obj)>`.

The type trait keeps the established `rebind_t` name, mirroring `std::simd` and `allocator_traits::rebind_alloc / rebind_alloc_t`. The value-producing operation takes the `_cast` suffix to make explicit that it is a named, type-directed, value-producing conversion, like `bit_cast`, `saturating_cast`, and `dynamic_pointer_cast`.

These facilities work together to enable uniform generic code:

```
template<typename Container>
auto widen_to_double(const Container& data) {
    return std::rebind_cast<double>(data);
}

// Works uniformly for all supported types:
std::vector<float> v = {1.0f, 2.0f, 3.0f};
auto vd = widen_to_double(v); // vector<double>

std::array<float, 3> a = {1.0f, 2.0f, 3.0f};
auto ad = widen_to_double(a); // array<double, 3>
```

The facilities are extensible via ADL, allowing user-defined types to participate in generic algorithms using the same interface.

§ 3. Supported Types

§ 3.1. Specification Principle

`rebind_cast` is provided for types where rebinding the element type produces a meaningful corresponding type. The presence of a `value_type` member typedef is a strong indicator that a type has a uniform element type, but it is neither necessary nor sufficient on its own:

- Types like `complex<T>` are clearly rebindable despite lacking `value_type`.
- Types like `std::stack<T>` have `value_type` but cannot be rebound without additional complexity (rebinding the underlying container).
- Associative containers have `value_type` but require also rebinding their comparator.

The determination of whether a type is rebindable depends on whether the rebinding operation is well-defined and produces a semantically equivalent structure with a different element type.

Standard library support is provided for:

1. **Sequence containers:** types with `value_type` representing a sequence of uniform elements.
2. **Scalar types with uniform components:** types like `complex<T>` that represent a single value composed of uniform components.
3. **User-defined types:** via ADL customisation for types where rebinding is appropriate.

§ 3.2. Standard Library Support

The following table shows the standard library types for which `std::rebind_cast` and `rebind_t` would be defined:

Input type	Has <code>value_type</code> ?	Result of rebinding to <code>double</code>
<code>std::array<T, N></code>	Yes	<code>std::array<double, N></code>
<code>std::vector<T, A></code>	Yes	<code>std::vector<double, rebind_alloc_t<A, double>></code>
<code>std::deque<T, A></code>	Yes	<code>std::deque<double, rebind_alloc_t<A, double>></code>
<code>std::list<T, A></code>	Yes	<code>std::list<double, rebind_alloc_t<A, double>></code>

§ 3.4. Excluded Types

Associative containers (`set`, `map`, etc.) are not supported because rebinding the element type requires also rebinding the comparator type, which lacks a general solution. See [§ 5.4 Associative Containers](#).

Tuple-like types (`tuple`, `pair`) are excluded because they are fundamentally heterogeneous types. See [§ 5.2 Tuple and Pair](#).

Container adaptors (`stack`, `queue`, `priority_queue`) are not included in initial support. See [§ 5.3 Container Adaptors](#).

Duration and units types (e.g., `std::chrono::duration`) are not included in the initial standard library support due to ambiguity about what should be rebound (see [§ 5.10 Duration and Units Types](#)), but such types may opt in via the ADL customisation mechanism if their authors define appropriate semantics.

§ 4. Examples

§ 4.1. Basic Usage

Converting element types is straightforward with `std::rebind_cast`:

```
// Arrays – preserves size
std::array<float, 4> af = {1.0f, 2.0f, 3.0f, 4.0f};
std::array<double, 4> ad = std::rebind_cast<double>(af);

// Type computation
using ArrayDouble4 = std::rebind_t<double, decltype(af)>;
static_assert(std::is_same_v<ArrayDouble4, std::array<double, 4>>);

// Vectors – preserves (rebound) allocator
std::vector<int> vi = {1, 2, 3, 4, 5};
std::vector<long> vl = std::rebind_cast<long>(vi);

// Complex numbers
std::complex<float> cf{3.0f, 4.0f};
std::complex<double> cd = std::rebind_cast<double>(cf);
```


§ 4.2. Generic Type Conversion

Structure-preserving type conversion works uniformly across container types:

```
template<typename U, typename Container>
auto convert_elements(const Container& c) {
    return std::rebind_cast<U>(c);
}

std::array<int, 5> ints = {1, 2, 3, 4, 5};
auto doubles = convert_elements<double>(ints); // array<double, 5>

std::vector<float> floats = {1.0f, 2.0f};
auto longs = convert_elements<long>(floats);    // vector<long>
```

§ 4.3. Complex Numbers

`rebind_cast` works uniformly for `complex`:

```
// Complex number precision conversion
std::complex<float> cf{3.0f, 4.0f};
auto cd = std::rebind_cast<double>(cf); // complex<double>{3.0, 4.0}

// Useful for mixed-precision algorithms
template<typename T>
auto high_precision_norm(const std::complex<T>& c) {
    auto hp = std::rebind_cast<long double>(c);
    return std::abs(hp); // Computed in higher precision
}
```

§ 4.4. User-Defined Types

Users can extend `rebind_cast` to their own types via ADL:

```
namespace mylib {
    template<typename T>
    struct Vec3 {
        using value_type = T; // Strong hint that rebinding is allowed.
    }
}
```

```

    T x, y, z;
};

// Provide rebind_cast via ADL
template<typename U, typename T>
Vec3<U> rebind_cast(const Vec3<T>& v) {
    return Vec3<U>{
        static_cast<U>(v.x),
        static_cast<U>(v.y),
        static_cast<U>(v.z)
    };
}
}

// Now Vec3 works with generic code:
mylib::Vec3<float> vf{1.0f, 2.0f, 3.0f};
auto vd = convert_elements<double>(vf); // Vec3<double>

// And rebind_t works automatically:
using Vec3d = std::rebind_t<double, mylib::Vec3<float>>;
static_assert(std::is_same_v<Vec3d, mylib::Vec3<double>>);

```

§ 5. Design Alternatives

§ 5.1. Range Adaptor Syntax

`std::rebind_cast` could support range adaptor syntax to enable composition with other range adaptors:

```
auto result = input | std::rebind_cast<double> | std::views::take(10);
```

This would follow the pattern of modern C++ range adaptors and enable natural pipelines. However, this functionality is deferred to future work to keep the initial proposal focused on core functionality.

§ 5.2. Tuple and Pair

`std::tuple` and `std::pair` are excluded because they are fundamentally heterogeneous types, even when all their element types happen to be the same.

Consider:

```
std::tuple<int, int, int> t = {1, 2, 3};
std::pair<int, int> p = {1, 2};

auto t2 = std::rebind_cast<double>(t); // What does this mean?
auto p2 = std::rebind_cast<double>(p); // Which elements to rebind?
```

The problem is not whether the types are currently the same, but that tuples (including `pair`, which is a 2-element tuple) are designed to hold potentially different types at each position. Without additional semantic constraints, the meaning of "rebind" is ambiguous for tuple-like types.

Users who need to convert tuple or pair elements can explicitly construct the target type:

```
std::pair<int, int> p = {1, 2};
std::pair<double, double> pd{static_cast<double>(p.first),
                             static_cast<double>(p.second)};
```

§ 5.3. Container Adaptors

Container adaptors (`std::stack`, `std::queue`, `std::priority_queue`) wrap underlying containers and present implementation challenges: the underlying container type is a template parameter that must itself be rebound, and the rebinding operation may need to traverse the adaptor's interface to extract elements.

This proposal excludes container adaptors from initial support. They can be added in future revisions once implementation experience is gained. Users who need to rebind adapted containers can rebind the underlying container type explicitly.

§ 5.4. Associative Containers

`std::set`, `std::map`, and related associative containers are excluded from this proposal because rebinding their element type requires also rebinding their comparator type, which lacks a general solution.

Consider:

```
std::set<float, MyFloatCompare> s = /*...*/;  
auto s2 = std::rebind_cast<double>(s); // What comparator should s2 use?
```

The result type would need to be `std::set<double, ???>`. Options include:

1. Use `std::less` — loses the custom comparator, changes semantics.
2. **Attempt to rebind the comparator** — no general mechanism exists for this; comparators may not be templated.
3. **Require users to provide the comparator** — defeats the purpose of `rebind_cast` being simple.

None of these solutions is satisfactory. The comparator-rebinding problem is orthogonal to element-type rebinding and requires separate consideration. For now, associative containers are excluded.

A future proposal could address this by:

- Defining a `rebind_comparator` mechanism for comparators.
- Extending `rebind_cast` to accept optional comparator arguments.
- Restricting support to comparators known to be rebindable (e.g., `std::less`).

Such extensions are beyond the scope of this proposal.

§ 5.5. Naming

§ 5.5.1. Consistency with the established `_cast` family

C++ has a well-established family of named, type-directed, value-producing conversions whose names end in `_cast`. The shared characteristic is that each takes an explicit target type and produces a value of a related type from its argument. In the core language:

- `static_cast`, `dynamic_cast`, `const_cast`, `reinterpret_cast`.

This family has continued to grow and is the convention for new function-style type-directed conversions:

- `std::bit_cast` (C++20) — type-directed reinterpretation between trivially-copyable types.

- `std::saturating_cast` (C++26, [P0543R3]) — type-directed numeric conversion with saturation semantics.
- The smart-pointer cast family: `std::static_pointer_cast`, `std::dynamic_pointer_cast`, `std::const_pointer_cast`, `std::reinterpret_pointer_cast`.
- The chrono cast family: `std::chrono::duration_cast`, `std::chrono::time_point_cast`.

`rebind_cast<U>(obj)` matches the family's invocation pattern precisely:

```
auto y = std::bit_cast<U>(x);           // C++20: reinterpret bits as U
auto y = std::saturating_cast<U>(x);    // C++26: numeric conversion of x with
auto y = std::rebind_cast<U>(x);        // rebind x's structure to use element
```

A new function-style, type-directed conversion in C++29 should join this family. Adopting `_cast` is the consistent choice.

§ 5.5.2. Comparison with other candidate names

Candidate	Verdict	Rationale
<code>rebind</code>	Rejected	Departs from the <code>_cast</code> convention for new function-style type-directed conversions, and conflates the type alias and the operation under a single name.
<code>rebind_cast</code>	Proposed	Joins the established <code>_cast</code> family; pairs naturally with <code>rebind_t</code> under the <code>_t</code> / <code>_cast</code> convention; preserves the <code>rebind</code> root, inheriting vocabulary from <code>rebind_alloc_t</code> and SIMD <code>rebind_t</code> .
<code>element_cast</code>	Rejected	Awkward for non-container types like <code>std::complex</code> (no obvious "element"); loses the <code>rebind</code> root vocabulary.
<code>value_cast</code>	Rejected	Suggests conversion of a single value; misses the structural aspect (allocator rebinding, container reconstruction); loses the <code>rebind</code> root vocabulary.
<code>structural_cast</code>	Considered	Accurate but heavy; coins a new term; less consistent with the compact names in the <code>_cast</code> family.
<code>recast</code>	Rejected	Vague; loses the <code>rebind</code> vocabulary; not in the <code>_cast</code> family.

Candidate	Verdict	Rationale
<code>convert</code>	Rejected	Too generic; doesn't convey the type-directed aspect; not in the <code>_cast</code> family.

§ 5.5.3. Disambiguation from `std::simd::rebind`

As a minor secondary benefit, the `_cast` suffix also avoids any potential confusion with `std::simd::rebind` (introduced in C++26). That trait lives in a different namespace and is a different kind of entity (a type-level alias template, not a value-producing operation), so there is no technical clash to resolve and no ambiguity at the language level. But because the existence of the SIMD-namespace trait was the original prompt for considering the `_cast` suffix, it is worth recording that this is a side benefit, not the driving motivation.

The choice of `rebind_cast` would be equally appropriate even if no `rebind`-named entity existed elsewhere in the standard library: the `_cast` and `_t` consistency arguments above stand on their own.

The earlier C++03 nested `rebind` member of `std::allocator` was deprecated in C++17 and removed in C++20 and is not relevant here.

§ 5.6. Customisation Mechanism

The proposal uses a customisation point object with ADL lookup, following the precedent of `std::ranges::swap` and other customisable standard library operations. We also provide the `rebind_t` type trait for use in compile-time contexts. The ADL hook is named `rebind_cast` (matching the user-facing CPO), so user types provide:

```
namespace user_ns {
    template<typename U, typename T>
    auto rebind_cast(const UserType<T>&) -> UserType<U>;
}
```

§ 5.7. Relationship Between `rebind_t` and `rebind_cast`

The type trait `rebind_t` is defined in terms of the function `rebind_cast`:

```
template<typename U, typename T>
```

```
using rebind_t = decltype(rebind_cast<U>(declval<T>()));
```

This ensures consistency: the type trait always produces the same type that the operation returns. An alternative would be to define them independently, but that creates potential for inconsistency and increases the customisation burden on user types (they would have to specialise both). Defining the trait in terms of the operation means users only need to provide one ADL hook.

§ 5.8. Relationship with `std::simd::rebind_t`

C++26 introduces `std::simd::rebind_t<U, basic_vec<T, Abi>>`, a SIMD-specific type-level alias that maps a SIMD type to the corresponding SIMD type with a different element type. Once `std::rebind_t` is added by this proposal, every `basic_vec` specialisation will be reachable by *two* spellings of essentially the same idea:

```
// Route A: SIMD-native trait (existing in C++26).
using A = std::simd::rebind_t<float, std::simd::basic_vec<int, Abi>>;

// Route B: generalised trait proposed by this paper.
using B = std::rebind_t<float, std::simd::basic_vec<int, Abi>>;
```

This is not a problem since the two names live in different namespaces, and would produce the same result type for SIMD inputs. However, it does need to be explicitly specified, so this subsection records the relationship and the equivalence guarantee.

§ 5.8.1. Coexistence

Both spellings continue to exist. We do *not* propose to remove or deprecate `std::simd::rebind` / `std::simd::rebind_t`:

- They are part of C++26 and are about to ship; deprecating them in C++29 with no usage-experience cycle would be premature.
- Source-compatibility for early adopters of `std::simd` matters.
- The two spellings serve subtly different purposes: `std::simd::rebind_t` is the right tool when code is *specifically* written against `std::simd`; `std::rebind_t` is the right tool for *generic* code that works uniformly across containers, `complex`, SIMD, and user-defined types.

§ 5.8.2. Equivalence guarantee

For any specialisation of `std::simd::basic_vec<T, Abi>`, this proposal requires:

```
std::rebind_t<U, std::simd::basic_vec<T, Abi>> denotes the same type as  
std::simd::rebind_t<U, std::simd::basic_vec<T, Abi>>.
```

This is a portability guarantee that users can rely on in concept constraints, partial specialisations, and overload resolution. It is *not* a coincidence to be observed by inspection.

The cleanest way to enforce the guarantee in the standard library wording is to specify the SIMD overload of `rebind_cast` so that its return type is `std::simd::rebind_t<U, std::simd::basic_vec<T, Abi>>`:

```
// Standard library overload for std::simd::basic_vec:  
template<class U, class T, class Abi>  
constexpr std::simd::rebind_t<U, std::simd::basic_vec<T, Abi>>  
    rebind_cast(const std::simd::basic_vec<T, Abi>& x);
```

Since `std::rebind_t<U, T>` is itself defined as `decltype(rebind_cast<U>(declval<T>()))` (see §5.7 Relationship Between `rebind_t` and `rebind_cast`), this specification chain makes the equivalence hold *by construction*: `std::rebind_t<U, basic_vec<T, Abi>>` is, by definition, whatever the SIMD overload of `rebind_cast` returns, which is by definition `std::simd::rebind_t<U, basic_vec<T, Abi>>`.

§ 5.8.3. Customisation goes through `rebind_cast`, not via trait specialisation

Because `std::rebind_t` is an alias defined in terms of `rebind_cast`, users (including future SIMD ABIs) should not attempt to extend rebinding behaviour by specialising `std::rebind_t` or `std::simd::rebind_t` directly. The supported customisation path is to provide an ADL `rebind_cast` overload for the type concerned. The trait will then automatically reflect the operation's behaviour.

This avoids the risk of drift between the trait and the operation, and keeps a single point of customisation.

§ 5.8.4. Guidance for users

- In **generic code** that may operate on containers, `std::complex`, SIMD types, or user-defined types: prefer `std::rebind_t` and `std::rebind_cast`.
- In **SIMD-specific code** that is statically constrained to `std::simd::basic_vec`: either spelling is correct; `std::simd::rebind_t` may be the more idiomatic choice for code that is otherwise written against `std::simd::*` names.
- In all cases the two spellings are guaranteed to denote the same type for SIMD inputs.

§ 5.9. Value-Preserving Conversions

This proposal does not impose special requirements for value-preserving conversions beyond standard C++ conversion rules. The element-wise conversion from `T` to `U` uses standard conversion semantics (equivalent to `static_cast<U>(t)` for each element).

```
std::vector<double> vd = {3.14, 2.71};  
auto vi = rebind_cast<int>(vd); // OK - explicitly requested narrowing
```

This follows the precedent of `std::simd`, which uses constructor explicitness to control conversion safety:

- Non-explicit constructors allow implicit conversions (i.e., conversion must be widening/value-preserving).
- Explicit constructors permit narrowing conversions.

For `rebind_cast`, the explicit, named cast call site is the user's signal that the conversion is intended, including narrowing conversions. This matches the precedent of `static_cast`, `bit_cast`, and `saturating_cast`: the named cast itself is the opt-in.

An alternative would be to require value-preserving conversions by default and provide a `flag_convert` parameter (following `simd`'s pattern for range load/store operations) to opt into narrowing:

```
std::rebind_cast<int>(vd); // Error - narrowing  
std::rebind_cast<int>(vd, std::flag_convert); // OK - explicit opt-in
```

However, this adds complexity without clear benefit:

- `rebind_cast` is always an explicit, named cast, so the user has already opted in.
- For generic code, the caller controls what conversions are valid.
- Standard C++ conversion rules already provide appropriate diagnostics (e.g., for explicit constructors).
- User-defined types can use `explicit` constructors to control conversion safety.

The current design is simpler and follows the principle that explicit cast calls indicate explicit intent — exactly as for `bit_cast` and `saturating_cast`.

§ 5.10. Duration and Units Types

Types representing quantities with units, such as `std::chrono::duration` or quantity types from units proposals like [P3045R1], are not included in the initial standard library support for this proposal.

Such types present ambiguity about what should be rebound:

- For `std::chrono::duration<Rep, Period>`, rebinding could mean changing the representation type (`Rep`) or the units (`Period`).
- The semantics of rebinding are unclear without additional context.

Additionally, `std::chrono::duration_cast` already provides conversion functionality for duration types, and is itself already a member of the `_cast` family that this proposal joins.

However, the ADL customisation mechanism allows such types to provide their own `rebind_cast` support if appropriate semantics can be established. Future proposals (including units proposals) can define `rebind_cast` for duration/units types if a suitable interpretation is agreed upon.

§ 6. Implementation Experience

`rebind_cast` has been prototyped for `std::basic_vec` (case 5 of §7.2.3 Customisation point object `rebind_cast`) within an experimental `std::simd` codebase.

The remaining standard-library cases (`array`, `vector`, `deque`, `list`, `forward_list`, `complex`) and the ADL hook for user-defined types are mechanical extensions; a reference implementation covering some of these is available at: <https://godbolt.org/z/aqK469x9e>.

§ 7. Wording

The wording below is provided to support LEWG’s design review.

§ 7.1. Header `<utility>` synopsis additions

The following declarations are added to the synopsis of `<utility>`:

```
namespace std {

    // [utility.rebind], generalised element-type rebinding

    // Type alias trait.
    template<class U, class T>
    using rebind_t = decltype(rebind_cast<U>(declval<T>()));

    // Customisation point object.
    // (Implementations may realise this as a function template, a function
    // object, or any other mechanism; see [[#wording_rebind_cast_cpo]] Note
    template<class U, class T>
    constexpr /* see below */ rebind_cast(T&& t);

}
```

§ 7.2. [utility.rebind] Generalised element-type conversion

This subclause specifies the type alias `std::rebind_t` and the customisation point object `std::rebind_cast`, which together provide a uniform interface for converting the element type of containers, scalar uniform-element types, SIMD vectors, and user-defined uniform-element types.

§ 7.2.1. General

`std::rebind_cast<U>(E)` produces a new value whose structure mirrors that of `E`, with each constituent element converted from its source type to `U` using ordinary C++ conversion semantics.

`std::rebind_t<U, T>` is the type of that value when `E` has type `T`.

The two facilities are linked by definition: the trait is specified in terms of the operation, ensuring that any type for which `rebind_cast<U>` is well-formed is automatically supported by `rebind_t<U,`

`T>`, and that the trait and the operation cannot disagree.

§ 7.2.2. Type alias `rebind_t`

```
template<class U, class T>
using rebind_t = decltype(std::rebind_cast<U>(declval<T>()));
```

Mandates: `std::rebind_cast<U>(declval<T>())` is a well-formed expression.

[*Note 1:* `rebind_t<U, T>` is therefore SFINAE-friendly: in a substitution context, it is well-formed if and only if `std::rebind_cast<U>` is well-formed for an argument of type `T`. This makes `rebind_t` usable in concept constraints, e.g. `requires { typename rebind_t<U, T>; }`. — *end note*]

[*Note 2:* For any specialisation of `std::basic_simd<T, Abi>`, `std::rebind_t<U, std::basic_simd<T, Abi>>` denotes the same type as `std::simd::rebind_t<U, std::basic_simd<T, Abi>>`. — *end note*]

§ 7.2.3. Customisation point object `rebind_cast`

The name `rebind_cast` denotes a customisation point object ([`customization.point.object`]). For a type `U` and a subexpression `E` whose type, after removal of cv-ref qualification, is `T`, the expression `std::rebind_cast<U>(E)` is expression-equivalent to:

1. an object of type `std::array<U, N>` whose *i*-th element equals `static_cast<U>(E[i])`, if `T` is `std::array<V, N>` for some type `V` and constant `N`, and each `static_cast<U>(E[i])` is well-formed; otherwise
2. an object of type `std::vector<U, allocator_traits<A>::template rebind_alloc<U>>` whose elements are, in order, `static_cast<U>(e)` for each element `e` of `E`, if `T` is `std::vector<V, A>` for some `V` and `A`, and the corresponding rebound allocator and element conversions are well-formed; otherwise
3. analogously for `std::deque<V, A>`, `std::list<V, A>`, and `std::forward_list<V, A>`: the result type is the corresponding container template with first argument `U` and second argument `allocator_traits<A>::template rebind_alloc<U>`, and the elements are `static_cast<U>(e)` for each element `e` of `E`, in order; otherwise
4. `std::complex<U>(static_cast<U>(E.real()), static_cast<U>(E.imag()))`, if `T` is `std::complex<V>` for some `V` and the conversions are well-formed; otherwise

5. an object of type `std::simd::rebind_t<U, T>` whose *i*-th lane equals `static_cast<U>(E[i])`, if *T* is a specialisation of `std::basic_simd`; otherwise
6. `rebind_cast<U>(E)`, where the meaning of the unqualified name `rebind_cast` is established as if by performing argument-dependent lookup ([basic.lookup.argdep]) only, treating `rebind_cast` as a function template that takes one explicit type-template argument followed by a function-call-argument list, if that lookup finds at least one declaration and the resulting call is well-formed; otherwise
7. `std::rebind_cast<U>(E)` is ill-formed.

Effects: The result is a new object whose elements (or component values) are the conversions, by `static_cast<U>`, of the corresponding elements of *E*.

Exception safety: If any element conversion exits via an exception, the call to `std::rebind_cast<U>(E)` exits via that exception. *E* is unaffected. No allocation performed during the construction of the result (for cases 2 and 3) leaks: any partially-constructed result is destroyed in the usual manner before the exception propagates.

[*Note 1:* The trait `rebind_t<U, T>` (see §7.2.2 [Type alias rebind_t](#)) is well-formed exactly when `std::rebind_cast<U>` applied to a value of type *T* is well-formed, by definition. — *end note*]

[*Note 2:* This specification deliberately does not name any implementation-detail namespace. Implementations are free to realise the dispatch using any mechanism — for example a function object, hidden in-namespace overloads, or `tag_invoke` — provided the observable expression-equivalences above are preserved. Standard-library types added in future revisions are supported by amending the list of cases above. User-defined types hook in via the ADL fallback (case 6). — *end note*]

[*Note 3:* Element conversion uses `static_cast<U>(e)`. If *U* is constructible from the element type via an explicit constructor, that constructor participates; if narrowing occurs, it is the user's choice (the named cast is itself the explicit opt-in). See §5.9 [Value-Preserving Conversions](#) for design discussion. — *end note*]

§ 7.2.4. Relationship with `std::simd::rebind_t`

For any specialisation `T = std::basic_simd<V, Abi>`:

- `std::rebind_t<U, T>` and `std::simd::rebind_t<U, T>` denote the same type.
- The result of `std::rebind_cast<U>(E)` for an *E* of type *T* has type `std::simd::rebind_t<U, T>`, by the wording of §7.2.3 [Customisation point object rebind_cast](#) case 5.

This guarantee is normative: programs may rely on it in concept constraints, partial specialisations, and overload resolution.

[*Note:* See [§5.8 Relationship with `std::simd::rebind\_t`](#) for the design rationale and guidance on which spelling to use in which context. — *end note*]

§ 7.2.5. Customisation by user-defined types

A user-defined type may opt into `std::rebind_cast` support by providing, in an associated namespace ([`basic.lookup.argdep`]) of the type, a function template named `rebind_cast` taking one explicit type-template argument followed by a function-call-argument list. Such a function is found via case 6 of [§7.2.3 Customisation point object `rebind\_cast`](#).

The provided function shall return a value whose type is the corresponding rebound type for the user's type, and whose constituent elements (or components) are obtained by element-wise conversion from those of the source value, in a manner consistent with the meaning of the user's type.

[*Example:*

```
namespace mylib {
    template<class T>
    struct Vec3 {
        using value_type = T;
        T x, y, z;
    };

    template<class U, class T>
    Vec3<U> rebind_cast(const Vec3<T>& v) {
        return Vec3<U>{ static_cast<U>(v.x),
                        static_cast<U>(v.y),
                        static_cast<U>(v.z) };
    }
}

mylib::Vec3<float> vf{1.f, 2.f, 3.f};
auto vd = std::rebind_cast<double>(vf);           // OK, mylib::Vec3<double>
static_assert(std::is_same_v<std::rebind_t<double, mylib::Vec3<float>>,
                          mylib::Vec3<double>>);
```

— *end example*]

A user-defined type shall not provide an overload of `rebind_cast` that, when found via case 6, returns a value of a standard-library type for which a higher-priority case in §7.2.3 Customisation point object `rebind_cast` is also applicable; the behaviour of programs that violate this is unspecified. [Note: This rule prevents users from accidentally redefining the meaning of `rebind_cast` for standard-library types. — *end note*]

§ 7.3. Feature-test macro

Add to [version.syn]:

```
#define __cpp_lib_rebind_cast YYYYMML // also in <utility>
```

The value `YYYYMML` is to be assigned by the editor at the time of adoption.

§ References

§ Informative References

[P0543R3]

Jens Maurer. *Saturation arithmetic*. 19 July 2023. URL: <https://wg21.link/p0543r3>

[P3045R1]

Mateusz Pusz, Dominik Berner, Johel Ernesto Guerrero Peña, Charles Hogg, Nicolas Holthaus, Roth Michaels, Vincent Reverdy. *Quantities and units library*. 22 May 2024. URL: <https://wg21.link/p3045r1>